

Day 124: Postman Fundamentals

1. Day number

Day 124

2. Topic name

Sending API Requests Using Postman

Today you will learn how to manually send API requests without writing Python or FastAPI code.

The main ideas are:

```
Postman
Request URL
HTTP method
Parameters
Headers
JSON body
Send
Response
```

3. Connection

Over the last few days, you learned the theory behind APIs.

Day 121

You learned the basic HTTP flow:

```
Client
  ↓ request
Server
  ↓ response
Client
```

Day 122

You learned HTTP methods:

```
GET    → Read
POST   → Create
PUT    → Update
DELETE → Delete
```

Day 123

You learned that requests and responses can contain:

Parameters
Headers
JSON
Status codes

Today, you will use **Postman** to actually construct and send those requests.

Conceptually:

```
graph TD
    You --> Postman
    Postman -- "HTTP request" --> API_Server[API Server]
    API_Server -- "HTTP response" --> Postman
    Postman --> You_inspect[You inspect the result]
```

Postman acts as the **client**.

4. Important topics

Today you will focus on:

- **Postman**
- **Request URL**
- **Method selection**
- **Parameters**
- **Headers**
- **JSON body**
- **Send button**
- **Response**

You will only perform basic manual testing.

No automation or scripting is needed today.

5. What is Postman?

Postman is a tool commonly used to work with APIs.

It allows you to create an HTTP request using a graphical interface.

Instead of writing code like:

```
send GET request to some URL
```

you can select:

GET

enter a URL:

`https://api.example.com/products/10`

and click:

Send

Postman sends the request for you.

Then it displays the server's response.

6. Foundational notes

Remember this from Day 121:

Client → Server

A browser can be a client.

A Python program can be a client.

A mobile application can be a client.

Today:

Postman = client

For example:

```
Postman
  |
  | GET /products/10
  ↓
API Server
  |
  | 200 OK
  |
  | JSON data
  ↓
Postman
```

Postman simply makes it easier for you to build and inspect these HTTP messages.

7. Why is Postman useful when learning APIs?

Suppose you are learning APIs and something is not working.

Without Postman, you might have several things happening at the same time:

```
Python code
API code
HTTP request
JSON
Server
Database
```

It can become difficult to know where the problem is.

Postman lets you focus specifically on the API request.

For example:

```
Method:
GET

URL:
/products/10
```

Click:

```
Send
```

Then immediately inspect:

```
Status code
Response body
Headers
```

This makes API concepts easier to see.

A useful beginner idea is:

Postman lets you manually test an API before writing client code for it.

8. Basic Postman screen mental model

When creating a simple request, think of Postman like this:

```
-----
Method      Request URL
[ GET ▼ ]   https://example.com/products/10
```

[Send]

Params | Headers | Body

Response

Status: 200 OK

```
{
  "id": 10,
  "name": "Keyboard",
  "price": 1200
}
```

The exact interface may vary slightly, but these are the important concepts.

9. Request URL

The **request URL** tells Postman where to send the request.

Example:

```
https://api.example.com/products/10
```

From previous lessons:

```
https://api.example.com
      ↓
server/domain

/products/10
      ↓
resource/path
```

If product 10 is being requested:

```
/products/10
      ↑
    product ID
```

10. Method selection

Before sending a request, you choose an HTTP method.

Postman normally provides a method selector near the request URL.

For example:

GET ▼

You might choose:

GET
POST
PUT
DELETE

From Day 122:

GET → Read
POST → Create
PUT → Update
DELETE → Delete

So if your goal is:

Read product 10

you would choose:

GET

and use:

/products/10

11. Parameters

Postman allows you to enter query parameters conveniently.

Suppose you want:

/products?category=electronics

You could think of:

Key	Value
category	electronics

Postman can construct the query portion of the URL for you.

Conceptually:

Base URL:
/products

Parameter:
category = electronics

Result:
/products?category=electronics

Remember from yesterday:

Path parameter
/products/10
 ↑
 identifies a resource

Query parameter
/products?category=electronics
 ↑
 filtering/options

12. Headers

Postman also provides a place to add HTTP headers.

For example:

Key	Value
Content-Type	application/json

From Day 123, remember:

Headers provide extra information about the HTTP message.

For JSON requests, you may see:

Content-Type: application/json

meaning:

"The body contains JSON."

You do not need to learn many headers today.

13. JSON body

Methods such as POST often need data to send to the server.

For example:

```
POST /products
```

The request body could contain:

```
{
  "name": "Keyboard",
  "price": 1200
}
```

In Postman, you would generally work in the **Body** area and enter the JSON data there.

Conceptually:

```
POST /products

Headers:
Content-Type: application/json

Body:

{
  "name": "Keyboard",
  "price": 1200
}
```

The body contains the actual product information.

14. The Send button

Once the request is prepared, Postman needs to actually send it.

You click:

```
Send
```

Conceptually:

```
Prepare request
↓
Click Send
↓
Postman sends HTTP request
↓
```

```
Server processes it
↓
Server returns HTTP response
↓
Postman displays response
```

The **Send** button is basically saying:

Send the HTTP request I have constructed.

15. The response

After sending the request, Postman displays the server's response.

For example:

```
Status: 200 OK
```

Response body:

```
{
  "id": 10,
  "name": "Keyboard",
  "price": 1200
}
```

From Day 123:

```
200 → success
201 → created
400 → bad request
401 → authentication problem
404 → not found
500 → server error
```

So when testing APIs with Postman, one of the first things you should inspect is:

```
Status code
```

Then inspect:

```
Response body
```

16. Easy GET request example

Suppose an API provides product information.

You want product 10.

Method

Select:

GET

URL

Enter:

`https://api.example.com/products/10`

Body

For this simple GET request:

No JSON body needed

Send

Click:

Send

Imagine the server returns:

200 OK

and:

```
{
  "id": 10,
  "name": "Keyboard",
  "price": 1200
}
```

The entire flow is:

```
Postman
  |
  | GET /products/10
  ↓
API Server
  |
  | Find product 10
  ↓
200 OK
```

```
{
  "id": 10,
  "name": "Keyboard",
  "price": 1200
}
```

You can now visually connect the theory from Days 121–123 with an actual API-testing tool.

17. What should you inspect after a GET request?

When the response arrives, start with two things.

First: status code

Example:

```
200 OK
```

Ask:

Did the request succeed?

Second: response body

Example:

```
{
  "id": 10,
  "name": "Keyboard",
  "price": 1200
}
```

Ask:

What data did the server return?

For today's exercise, these two pieces are enough.

18. Easy POST request example

Now imagine you want to create a new product.

Step 1: Select the method

Choose:

```
POST
```

Step 2: Enter the URL

```
https://api.example.com/products
```

Notice that we are not requesting one existing product.

We want to create a new resource in the products collection.

Step 3: Provide JSON

In the request body:

```
{  
  "name": "Mouse",  
  "price": 500  
}
```

Step 4: JSON information

The request conceptually indicates:

```
Content-Type: application/json
```

Step 5: Send

Click:

```
Send
```

Step 6: Inspect response

A successful creation might return:

```
201 Created
```

with:

```
{  
  "id": 11,  
  "name": "Mouse",  
  "price": 500  
}
```

Conceptually:

```
Postman  
  |  
  | POST /products  
  |  
  | {  
  |   "name": "Mouse",
```

```
|    "price": 500
|  }
↓
Server
|
| Create product
↓
201 Created

{
  "id": 11,
  "name": "Mouse",
  "price": 500
}
```

19. GET vs POST in Postman

Here is the main difference for today's lesson:

Feature	GET	POST
Main purpose	Read	Create
Select method	GET	POST
URL	Yes	Yes
JSON request body	Usually not for this basic example	Commonly yes
Click Send	Yes	Yes
Inspect response	Yes	Yes

Remember:

```
GET
↓
Ask server for data

POST
↓
Send data to create something
```

20. Problem statement

Your task today is deliberately small.

Create one simple **GET request** in Postman.

Imagine the API has an endpoint:

```
https://api.example.com/products/5
```

Your goal is to retrieve product 5.

In Postman:

Choose the correct HTTP method.

Enter the URL.

Send the request.

Inspect the HTTP status code.

Inspect the JSON response body.

Suppose the successful response is:

```
200 OK
```

with:

```
{
  "id": 5,
  "name": "Monitor",
  "price": 8000
}
```

Your job is to identify:

1. Which method did you select?
2. What URL did you enter?
3. Did this simple request need a JSON body?
4. What did you click to send the request?
5. What status code came back?
6. What data appeared in the JSON response?

Do not write Python code.

21. Concepts used

Today's exercise combines:

Postman
HTTP
Client
Server
Request
Response
HTTP method
URL
Parameters
Headers
JSON
Status code

You are essentially using Postman to visualize the concepts from the previous three days.

22. Thought process

Before sending anything, ask:

Step 1: What am I trying to do?

Example:

Retrieve product 5

That means:

Read

From Day 122:

Read → GET

So select:

GET

Step 2: Where should the request go?

Use:

<https://api.example.com/products/5>

That is your request URL.

Step 3: Do I need extra data?

For this simple GET example:

No JSON request body

Step 4: Send the request

Click:

Send

Step 5: Inspect the result

First:

Status code

Then:

JSON response

This should become your beginner API-testing habit.

23. Beginner-friendly Postman steps

For a simple GET request, use this workflow:

START

Open Postman

Create/open a request

Choose GET

Enter request URL

Check whether any parameters are needed

Check whether any headers are needed

For this basic GET:
do not add a JSON body

Click Send

Wait for the server response

Inspect status code

Inspect JSON response body

END

You can remember the shorter version:

```
Method
↓
URL
↓
Parameters / Headers / Body if needed
↓
Send
↓
Status code
↓
Response body
```

24. Suggested solving approach: manual API testing

For now, use Postman manually.

Do not automate anything.

For each request:

1. Decide what operation you want.
2. Choose the HTTP method.
3. Enter the URL.
4. Add parameters if required.
5. Add headers if required.
6. Add JSON body if required.
7. Click Send.
8. Read the status code.
9. Read the response.

This mirrors how you should think about APIs in general.

25. Easy edge case: wrong URL

Suppose the correct URL is:

```
https://api.example.com/products/5
```

but you accidentally enter:

```
https://api.example.com/prodcts/5
```

Notice:

```
products  
prodcts
```

The API may not recognize that endpoint.

Conceptually:

```
Postman  
  |  
  | GET /prodcts/5  
  ↓  
Server  
  |  
  | Unknown path  
  ↓  
Error response
```

You might receive something such as:

```
404 Not Found
```

So when an API request fails, always check the URL carefully.

26. Easy edge case: wrong method

Suppose you want to retrieve product 5.

The normal beginner mapping is:

```
Read → GET
```

But imagine you accidentally choose:

```
DELETE
```

You are no longer expressing the same intention.

```
GET /products/5
```

means approximately:

```
"Give me product 5."
```

while:

```
DELETE /products/5
```

means approximately:

```
"Delete product 5."
```

This is why you should always check the selected HTTP method before clicking **Send**, especially when working with real APIs.

27. A simple debugging checklist

If your beginner Postman request does not work, inspect these pieces:

```
Method correct?
  ↓
URL correct?
  ↓
Parameters correct?
  ↓
Headers correct?
  ↓
JSON body needed?
  ↓
JSON body valid?
  ↓
What status code came back?
```

For today, focus especially on:

```
Method
URL
Status code
Response body
```

28. Hint only

For the exercise, you want to:

retrieve product 5

Translate:

retrieve → read

Then remember:

Read → ?

Your choices from Day 122 are:

GET
POST
PUT
DELETE

After choosing the method:

enter URL
↓
click Send
↓
look at status code
↓
look at JSON response

Do not add a request body unless the API actually requires one.

Day 124 key takeaway

Your journey now looks like this:

Day 121
HTTP request/response
↓
Day 122
HTTP methods
↓
Day 123
Parameters, headers, JSON, status codes
↓
Day 124
Use Postman to send and inspect requests

The core Postman workflow is:

```
graph TD; A[Choose method] --> B[Enter URL]; B --> C[Add parameters if needed]; C --> D[Add headers if needed]; D --> E[Add JSON body if needed]; E --> F[Click Send]; F --> G[Inspect status code]; G --> H[Inspect response body];
```

And the most important idea is:

```
graph TD; A[Postman] -- "HTTP Request" --> B[API Server]; B -- "HTTP Response" --> C[Postman]; C --> D[You inspect it];
```

At this point, you are moving from **understanding HTTP conceptually** to **manually testing APIs**, which will make later FastAPI work much easier.